Introduction to Dynamic Vision Sensors

laboratory

Winter semester, 2024/2025

Piotr Wzorek, Tomasz Kryjak

PUBLISHED BY AGH

First printing, October 2022

Contents

1 — Introduction, understanding event data

1.1 Requirements, calculating the final grade

A point system will used during the laboratory. The total amount of 100 points is made up of points for the implementation of basic exercises (70 points) and extension exercises (30 points). On the basis of the total number of points, a grade will be assigned in accordance with the applicable Study Regulations at AGH.

The exam can be taken only after a positive mark has been obtained in the laboratory. The exam will have an oral form.

Final grade = 0.6 x laboratory grade + 0.4 exam grade

Attendance in the laboratory is obligatory. In the case of one unexcused absence, the student can receive a maximum grade of 4.0 (regardless of the number of points). In the case of two unexcused absences, the student may receive a maximum grade of 3.0 (regardless of the number of points). In the case of three or more unexcused absences, the student will not receive a credit.

1.2 Obligatory literature

G. Gallego, et al.,"Event-Based Vision: A Survey" in IEEE Transactions on Pattern Analysis & Machine Intelligence, vol. 44, no. 01, pp. 154-180, 2022. doi: 10.1109/TPAMI.2020.3008413 Link to paper

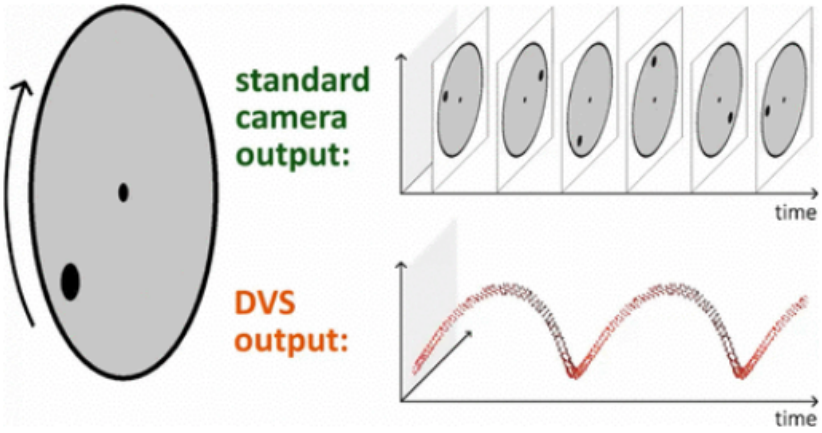
1.3 Theoretical part

1.3.1 What are event cameras?

An event camera (also known as Dynamic Vision Sensor – DVS) is a neuromorphic sensor that takes its inspiration from the human eye. Unlike classical cameras, which record the brightness (colour) level for a given pixel every specified time interval (frame per second parameter), a DVS records brightness changes independently (asynchronously) for individual pixels. Consequently, the data captured by the camera does not depend on the clock but the dynamics of the scene. As a result, a stream of events is available on the output, where each is described by 4 values:

$$e = \{t, x, y, p\} \tag{1.1}$$

where: t is the event capture time (timestamp), x and y are the pixel coordinates, and p is the value 0 or 1 corresponding to the event polarity (positive or negative change in pixel value reaching a fixed threshold).



Mueggler, Elias, Basil Huber, and Davide Scaramuzza. "Event-based, 6-DOF pose tracking for high-speed maneuvers." *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2014.

Figure 1.1: Visualization of how the event camera works

1.3.2 Why do We use event cameras?

Such an approach has a number of interesting consequences. First, a single pixel consists of an analogue and a digital part. Changes are detected in the analogue part, which results in low latency (fast response time) and energy efficiency (insufficient change means that the digital part will not be triggered). In addition, the analysis of the logarithm of the change in brightness of each particular pixel independently allows one to obtain a high dynamic range (above 120 dB). Consequently, DVS sensors are relatively resistant to large differences in the brightness of the recorded scene, which for classical cameras becomes a problem (part of the image is overexposed or too dark). This property also enables the correct detection of objects even under very high or limited light conditions.

Second, the transmission of information only about the occurring changes eliminates the usually unwanted redundancy (a traditional camera sends information about the status of all pixels, even those whose brightness has not changed). Another characteristic of event cameras is their high sampling rate – the timestamp has a resolution of microseconds. In addition, the camera data stream can reach up to 1200 MEPS (million events per second) depending on the chip and hardware interface used. The limitation of redundant information and the high frequency enable the processing of key information even under conditions of high variability, i.e. when the objects registered by the camera move in relation to it at high speed.

1.4 Practical part

During this class we will use Python and DAVIS 240C dataset. Read about it on this website: `https://rpg.ifi.uzh.ch/davis_data.html`. You can download dataset yourself with this link: `https://rpg.ifi.uzh.ch/datasets/davis/shapes_rotation.zip`

Basic Exercise 1.1 Visualising Event Data. ■

1. Open (extract) the dataset folder from `/home/Downloads/shapes_rotation`.
2. Examine images from `images` folder – this is a scene recorded with an event camera.
3. Open the `events.txt` file, analyze the data format. What are the particular data values ?
4. Create a Python file (use Visual Studio Code). Save it to the same folder, where the `events.txt` is located.
5. Read `events.txt` and save it to a variable (if you do not know how, check "google").
6. Now we will analyse the file line by line:
   - Each file line should be saved as a separate list member.
   - Event values (timestamp, coordinates and polarity) should be splitted into vectors (`split` function could prove useful).

   Attention!
   Save only those events, where timestamp < 1. Saving all events would take a lot of time!

   To achieve this, there are different options possible. Check in "google" how the function `readline` works. It could be used in a `while` loop to iterate over the whole file. Use the `split` function to separate the column is a given line. To limit the number of samples, check if the timestamp is > 1 (if so – break the loop). In the end we should have a list o events.
7. In the next step we should split the events into particular variables: timestamp, x, y and polarity. Create 4 empty lists, do a for loop over the created events and extract the information. Again. If you do not know how – check how to iterate over a list in Python.
8. Analyze the events:
   - Print number of events.
   - Print first and last timestamp.
   - Print maximum and minimum values of pixel coordinates (x and y, respectively) and compare it to image resolution of 240x180.
   - Which events are there more of? Those with positive or negative polarity?

   Use the print function. Provide a description for each filed. If you do not know .... :) For the polarity you can sum the the whole list to get the number of positive ones and for the negatives subtract the number of positive from the total.
9. Visualize event data with 3D chart:
   - Chart dimensions should be x,y coordinates and timestamp.
   - Positive and negative events should have different colors.
   - The chart should have labels for each dimension.
   - Show the source code and the generated chart to your teacher.

   Protip
   `scatter3D` function and `matplotlib` library could prove useful.

   Hints. You have to split the data into positive and negative lists (so in total 6 list). The scatter plot is described well in the documentation.
   Analyse the data.

Basic Exercise 1.2 Analysing Event Data ■

1. Plot 3D chart (as in exercise 1.1) but only for the first 8000 events.

- Rotate the chart so that you can see the object's shapes clearly.
- Compare the rotated chart to the first image from the `images` folder.

2. Plot the 3D chart but only for events with timestamp between 0.5 and 1
   - Rotate the chart so that you can see the direction of movement of objects over time.
3. Answer following questions (as comments in Your code):
   - How long is the sequence used during exercise 1.1 (in seconds)?
   - What's the resolution of event timestamps?
   - What does the time difference between consecutive events depend on?
   - What does positive/negative event polarity mean?
   - What is the direction of movement of objects in exercise 1.2?
4. Show the source code and the generated charts to your teacher.

Extension Exercise 1.1 Verify your conclusions ■

The dataset used in the class includes sequence, where all objects are stationary relative to each other. Thus, we can determine the direction of movement of the entire scene relative to the camera (for events with timestamps from 0.5 to 1). Determine the direction (in x and y planes) of movement of the objects and the distance by which they were moved (in pixels) – this is a form of optical flow computation. Compare the result to the answer to question 5 from task 1.2.

Protip
Calculate the average values of pixel coordinates over short time intervals (e.g., 0.01 s) and display their changes on a 3D graph. Analyze the obtained data, calculate the difference between values (consecutive).

.

2 — Event-frame representations

2.1 Obligatory literature

Wzorek, Piotr, and Tomasz Kryjak. "Traffic Sign Detection With Event Cameras and DCNN." arXiv preprint arXiv:2207.13345 (2022). Link to paper

2.2 Theoretical part

2.2.1 Why do we use event frame representations?

The subject of vision systems is one that has been developed for many years. So far, computer vision algorithms are based on classic image frames - 2D matrices representing the values (brightness) of individual pixels (it is not uncommon to use more channels, for example, three - R (red), G (green) and B (blue)).

The event camera data stream, which can be described as a sparse point cloud in space-time, presents significant challenges in terms of designing analysis and image processing algorithms. It is difficult to use them with state-of-the-art computer vision algorithms and the previous achievements of over 60 years of computer vision cannot be easily applied. Three main approaches are used: an attempt to analyse the data as a point cloud (in a way similar to, e.g. LiDAR data), transformation (accumulation) of event data into various types of event frames, and reconstruction. In today's class, we will focus on the representation of event data in the form of frames, which are analogous to traditionally used matrices containing pixels brightness value.

2.2.2 Event-frame representations

Various event data representations in the form of frame-matrices analogous to traditional video data are proposed in the literature. In general, the process involves aggregating event data over a given time window and then representing it as a 2D matrix. To define some of the representations, we use a following notation. We define a function Σ_e that, for each pair of event coordinates $u(x,y)$, matches the time of its event t (Equation (2.1)) and a function P_e that, for an event coordinate, matches its polarity (Equation (2.2)).

$$u : t = \Sigma_e(u), \Sigma_e : R^2 \rightarrow R \tag{2.1}$$

$$u : p = P_e(u), P_e : R^2 \rightarrow \{-1, 1\} \tag{2.2}$$

Additionally, τ was defined as the accumulation time of the event data (the representation is created through aggregation).

2.2.3 Event frame

The simplest form of event data representation is the "event frame". Each pixel is assigned a polarity value of the recorded events. Therefore, it takes into account only information about coordinates and polarisation of pixel brightness changes, ignoring temporal information. A representation defined in this way can be described as:

$$f(u,t) = \begin{cases} P_e(u) & for\ t - \Sigma_e(u) \in (0;\tau) \\ 0 & for\ t - \Sigma_e(u) \in (\tau;+\infty) \end{cases} \tag{2.3}$$

Any pixels for which no event have been registered are marked with the value 127, and events are marked as 0 or 255 depending on the polarity.

2.2.4 Exponentially decaying time surface

An extension of the idea of an event frame is a representation that also takes into account the time of occurrence of an event in a time window. This method assumes that the weight of information decreases exponentially with time to zero. This representation is denoted by the Equation (2.4).

$$f(u,t) = \begin{cases} P_e(u) * e^{\frac{\Sigma_e(u)-t}{\tau}} & for\ \Sigma_e(u) \leq t \\ 0 & for\ \Sigma_e(u) > t \end{cases} \tag{2.4}$$

2.2.5 Event frequency

The sigmoid representation uses information about the frequency of events for a given pixel, which is ignored by the approaches presented so far. It is defined as:

$$f(x) = 255 * \frac{1}{1+e^{-x/2}} \tag{2.5}$$

The sum of the polarisation values present in a given pixel is denoted as x.

2.3 Practical part

During this class we will use Python and DAVIS 240C dataset **(the same as last week)**. Read about it on this website: `https://rpg.ifi.uzh.ch/davis_data.html`. You can download dataset yourself with this link: `https://rpg.ifi.uzh.ch/datasets/davis/shapes_rotation.zip`

Basic Exercise 2.1 Event frame ■

1. Open (extract) the dataset folder from `/home/Downloads/shapes_rotation`.
2. Create a Python file (use Visual Studio Code). Save it to the same folder, where the `events.txt` is located.
3. Import `cv2` and `numpy` libraries - we are going to need those during this class.
4. Read `events.txt` and save it to a variable (as You did last week).
5. Analyse the file line by line, save values, split them into particular variables: timestamp, x, y and polarity. (Points 6-7 from Basic Exercise 1.1 with some minor changes).
   - Remember to convert strings to proper variable types - timestamps should be floating points, coordinates and polarities should be integers
   - During this exercise, it would be easier to represent polarity as {1,-1} instead of {1,0}. Update negative polarity values during splitting events into variables.

Attention!
Save only those events, where timestamp > 1 and < 2. Saving all events would take a lot of time! This interval was chosen because of the significant movement of objects relative to the camera, which will facilitate the task.

6. Implement function that creates event frame representation.
   - Define function *event_frame* with following input arguments - event coordinates lists, polarities list and image shape.
   - Inside the function, create 2D matrix with image shape. Initialize this matrix with value 127 (as mentioned in the Theoretical part of this class). This matrix will represent our image.

     Protip
     `numpy` python library could prove useful. Its function `np.ones()` can be used to create a matrix with ones as all its elements. This matrix can be later multiplied by 127.

   - In order to properly visualize matrix as image, change its type to uint8 (values are represented by integers of 0-255 where 0 is black and 255 is white).

     Protip
     You can do it with `image.astype(np.uint8)` function (if you imported numpy library first).

   - Loop through all events and set pixel values for each coordinates to 255 (for positive polarity) and to 0 (for negative polarity).
   - This function should return image, that is, the completed pixel matrix.
7. Each event frame represents the events that were recorded in a given time window. Thus, it is necessary to select the duration of data aggregation, which will then be used to create a representation. Create a variable `tau` and set it to 10 ms (remember that timestamp is expressed in seconds, so this variable should also be expressed in seconds).
8. Loop through all saved events. Create temporary lists to store aggregated event data. Append saved events to temporary lists untill the difference between the first and last timestamp is greater than tau. Then call the function you prepared earlier and pass the aggregated event data (temporary lists) - timestamps, coordinates and polarity - to it as arguments.
9. Show image with `opencv` functions.

   Protip
   All `opencv` functions are well documented - just google it. You should use `imshow()` function to show created image. You should also use `waitKey()` function just after `imshow()` - This ensures that the image will be displayed until you press any key on the keyboard.

10. After generating event frame you should keep looping through saved events in order to prepare next event frame. Clear temporary lists and keep aggregating data. The event frame should be generated every 10 ms. Since the length of entire sequence is 1s, you should get 100 images. Compare generated images to those in *images* folder.

    Protip
    You should use one *for* loop with *if* to verify the timestamp difference. Whenever the

condition is met, you call *event_frame*, clear temporary list and show image. Then - the loop can be continued as soon as you press any key on the keyboard.

11. Try running the same code with different values of `tau` (1ms, 10ms, 100ms). How does the aggregation time `tau` affect the generated event frames? Answer as comment to your code.
12. Show the source code and the generated images to your teacher.

Basic Exercise 2.2 Exponentially decaying time surface ■

1. Implement function that creates exponentially decaying time surface representation.
   - Define function *exponential_decay* with following input arguments - timestamps list, event coordinates list, polarities list, tau and image shape.
   - Inside the function, create 2D matrix with image shape. Initialize this matrix with zeros. This matrix will represent our image.

     Protip
     This time you should use `np.zeros()`.

   - Save the biggest timestamp to variable.
   - Loop through all events and set pixel values for each coordinates according to formula from chapter 2.2.4.

     Protip
     You can use `np.exp()` to calculate the exponential. Use the biggest timestamp as `t` in the formula and `tau` input argument as τ.

   - In order to properly visualize matrix as image you have to normalize it (so that the values of pixel will be stretched to 255-0). Then - change matrix type to uint8.

     Protip
     Create second matrix of the same shape and call it `normalizedImg`. Use `cv2.normalize()` function with `cv2.NORM_MINMAX`.

   - This function should return `normalizedImg`, that is, the completed and normalized pixel matrix.
2. Update the code from exercise 2.1 to call *exponential_decay* function. Create exponentially decaying time surface representations for different values of `tau` (1ms, 10ms, 100ms). Compare generated images to those in *images* folder.
3. Show the source code and the generated images to your teacher.

Basic Exercise 2.3 Event frequency ■

1. Implement function that creates event frequency representation.
   - Define function *event_frequency* with following input arguments - event coordinates list, polarities list, image shape.
   - Inside the function, create 2D matrix with image shape. Initialize this matrix with zeros. This matrix will represent our image.
   - Loop through all events and calculate the sum of the polarisation values present in each pixel. This values should be saved inside created matrix (under the respective coordinates).
   - Calculate values of pixels according to formula in chapter 2.2.5.

> **Protip**
> You can use entire matrix inside the `np.exp()` function. After calculating the sums of the polarisation values you can just implement the function with one line of code: `image = 255  (1 + np.exp(-image / 2))`

- In order to properly visualize matrix as image you have to normalize it (so that the values of pixel will be stretched to 255-0). Then - change matrix type to uint8.
- This function should return `normalizedImg`, that is, the completed and normalized pixel matrix.

2. Update the code from exercise 2.1 to call *event_frequency* function. Create event frequency representations for different values of `tau` (1ms, 10ms, 100ms). Compare generated images to those in *images* folder.
3. Show the source code and the generated images to your teacher.

Extension Exercise 2.1 Neighborhood suppression time surface ■

Implement function that generates Neighborhood suppression time surface representation as described in this paper - the formula and the pseudo-code are well documented. Update the code from exercise 2.1 to call *neighborhood_suppression* function. Create neighborhood suppression time surface representations for different values of `tau` (1ms, 10ms, 100ms).

3 — Image reconstruction

3.1 Obligatory literature

Scheerlinck, Cedric, et al. "Fast image reconstruction with an event camera." Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. 2020. Link to paper

3.2 Theoretical part

Last week we explored the problem of the unusual format of event data and its use for classical computer vision algorithms. We used various event frame generation methods to represent the data in a form analogous to traditional frames. In addition to aggregating events over a specific time window, another possibility is to use deep neural networks to reconstruct video frames from event data. An example is FireNet – a state-of-the-art fully convolutional recurrent neural network.

3.3 Practical part

During this class we will use Python and DAVIS 240C dataset **(the same as last week)**. Read about it on this website: `https://rpg.ifi.uzh.ch/davis_data.html`. You can download dataset yourself with this link: `https://rpg.ifi.uzh.ch/datasets/davis/shapes_rotation.zip`

Basic Exercise 3.1 FireNet image reconstrucation ■

1. Open `https://github.com/cedric-scheerlinck/rpg_e2vid/tree/cedric/firenet` repository, read `README`.
2. Clone repository on Your hard drive (use `git clone` function).
3. Checkout repository to `cedric/firenet` branch (use `git checkout` function).
4. Download pretrained FireNet neural network from here. Save to repository directory.
5. In order you use `events.txt` file for image reconstruction with FireNet source code You need to prepare it. First, add image width and height in the first line of this file (`240 180`). Then, compress this file with `zip`.

Attention!
Opening such a huge `txt` file with editor will take a lot of time. You should use either a program in Python that reads the previous file and writes a new one, or an editor in the Linux console (such as `vim`).

6. Run reconstruction as described in repository `README` file. Use following parameters with *run_reconstruction.py* script:
   - *-c path_to_pretrained_network*
   - *-i path_to_zip_file_with_events*
   - *–auto_hdr*
   - *–display*
   - *–show_events*
7. Analyze the meaning of all used parameters. Show the source code and the generated sequence to your teacher.

Basic Exercise 3.2 FireNet image reconstrucation with fixed duration ■

1. Run reconstruction as described in repository with fixed duration parameter. In the previous exercise we reconstructed images creating each frame from constant number of events. Consequently, the time between successive frames is variable - it depends on the difference between the first and last timestamp. Use following parameters with *run_reconstruction.py* script:
   - *-c path_to_pretrained_network*
   - *-i path_to_zip_file_with_events*
   - *–auto_hdr*
   - *–display*
   - *–show_events*
   - *–show_events*
   - *–fixed_duration*
   - *–window_duration value_in_ms*
2. Analyze the meaning of all used parameters. Run reconstruction for different window duration values - 1s, 100 ms, 10 ms, 1 ms. Compare the results and discuss it with comments. Show the source code, comments and the generated sequences to your teacher.

Extension Exercise 3.1 FireNet image reconstrucation accuracy score ■

Run reconstruction with fixed duration parameter and set *window_duration* so that the number of reconstructed frames for entire sequence is the same as number of images in *images* folder. Save reconstructed images to some folder (– *output_folder* parameter). Compare each of the reconstructed frames to the gray-scale images in the *images* folder. Calculate either a Mean Squared Error or Structural Similarity Measure (google it!). In addition, calculate the differences between the images for each frame - display the results achieved.

4 — Dataset generation for DCNN

4.1 Obligatory literature

Orchard, G.; Cohen, G.; Jayawant, A.; and Thakor, N. "Converting Static Image Datasets to Spiking Neuromorphic Datasets Using Saccades", Frontiers in Neuroscience, vol.9, no.437, Oct. 2015. Link to paper

4.2 Theoretical part

During the classes so far, we have dealt with various methods of representing event data in the form of frames. We will devote this and the next classes to preparing a classification system based on deep neural networks and event data. For this purpose, we will use a popular dataset - N-MINST. It contains correctly labeled handwritten numbers from 0 to 9. The task we will perform is to correctly classify numbers based on event data.

Neural networks used in the literature to classify images usually use data represented in the form of frames. To adapt a neural network to a given task, a training process must be carried out. For this to be possible, we need a large dataset with correctly labeled frames. In today's class, using the knowledge gained so far, we will generate various forms of event data representations for use in training the neural network.

4.3 Practical part

During this class we will use Python and N-MNIST dataset. Read about it on this website: `https://www.garrickorchard.com/datasets/n-mnist`. You can download dataset yourself with this link: `https://drive.google.com/drive/folders/16PYo5Jo3VlFC6-Lvw4c2hB-EAEf_egTL?usp=sharing`

Basic Exercise 4.1 Prepare dataset ■

1. Download dataset from link above. You'll need both `Train.zip` and `Test.zip` files.
2. Create folder `raw_data`. Extract downloaded files in this folder.
3. Examine extracted data. Each folder name will be used as class (it corresponds to handwritten number). Note, that event data is saved as binary files. You will not be able to open it with Text editor.

4. Create Python file and implement code that loops through all files in Train and Test folders and its subfolders. For each file print its name and path.

> **Protip**
> You can do it with `os` Python library (just Google it).

5. Print the number of all files in both folders. Compare number of Test and Train files.
6. Show the source code to your teacher.

Basic Exercise 4.2 Open dataset. ■

1. Download function that can be used to open bin file and save it as event from this link.
2. Examine downloaded function.
3. Use this function and code from exercise 4.1 to open each bin file. For each of them print number of all events and image width and height. Calculate and print average number of events in all files.
4. Show the source code to your teacher.

Basic Exercise 4.3 Generate frames dataset. ■

1. After implementing the above tasks, we have complete code that opens each binary file with event data in turn. We can use it to generate frames that will be applied to train the neural network.
2. On the UPEL platform there is a file with records for the type of representation. Each student should prepare a dataset from one representation type. In a week's time, each will prepare a neural network performing a classification task based on the chosen representation. Then - the obtained results will be compared among themselves. Sign up for one of the representations on UPEL. For each representation should be entered at least two, and a maximum of 3 people.
3. Use function from 2nd class to prepare selected representations for all event files. You should use a 50ms time windows (the dataset has timestamps expressed in us). Save generated frames.

> **Protip**
> You can do it with `cv2.imwrite()` method form Python library OpenCV (just Google it).

> **Attention!**
> Each generated file should have unique name. Moreover, folder structure should be preserved. Each frame generated from event file from folder `raw_data/Train/0` should be saved to `frame_data/Train/0` folder.

4. After completing this task, we have a ready generated database containing frames. Each frame is located in a folder whose name denotes its class. In a week's time, we will use this collection to train the neural network.

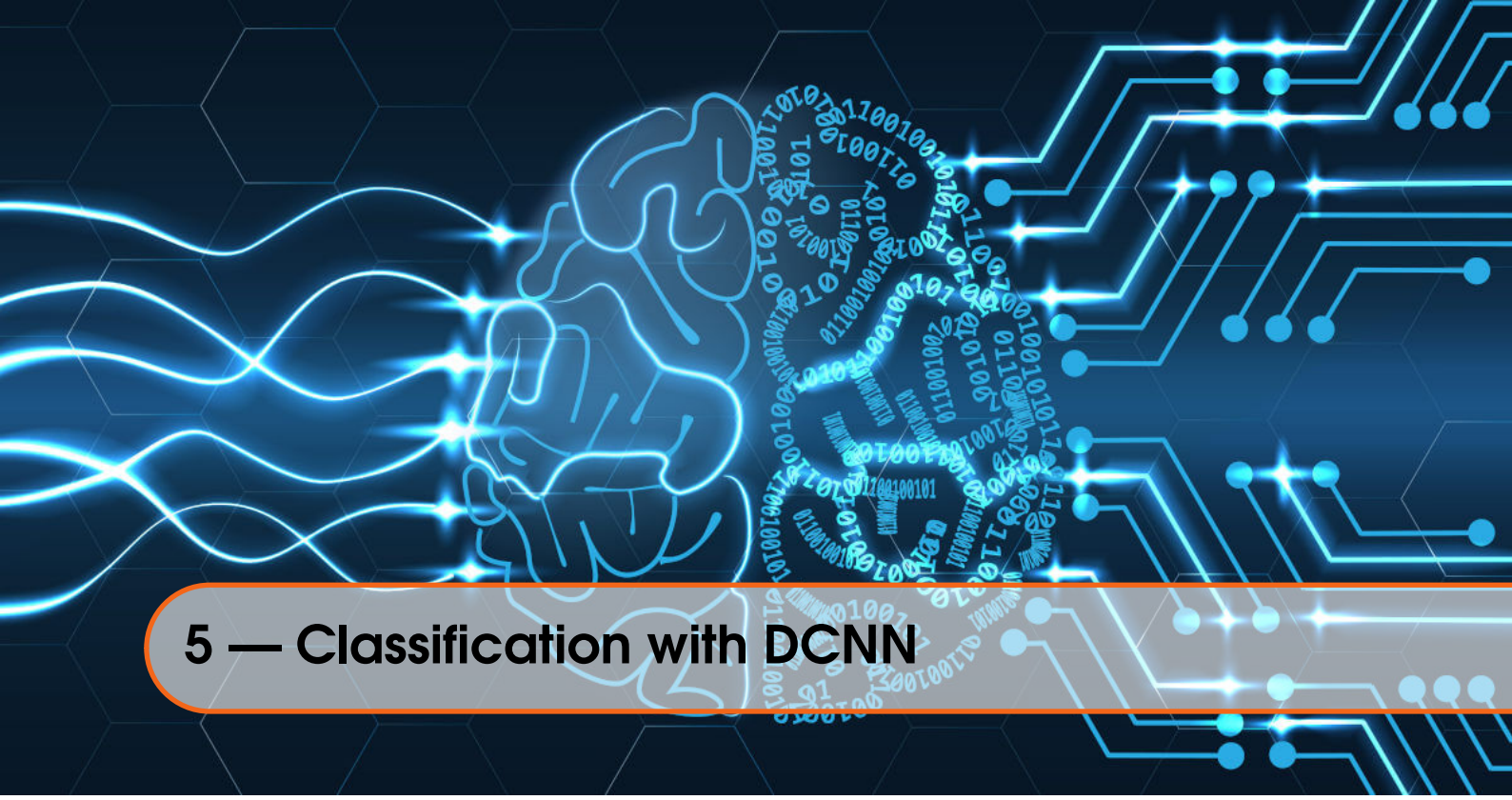
> **Attention!**
> One of the types of representation proposed on the UPEL platform is fusion. It refers to the fusion of the three considered representations as consecutive input channels. The motivation behind the proposed representation fusion was to use as much information as possible from the source event data (individual representations take into account other features of the event data). In order to prepare this type of representation, just use

all three functions from 2nd class and calculate three separate images. Than, create a image with 3 channels and use each representation as one of them.

```
image4 = np.zeros((width, height, 3))
image4[:,:,0] = image1
image4[:,:,1] = image2
image4[:,:,2] = image3
```

Extension Exercise 4.1 FireNet image reconstrucation dataset ■

Create N-MNIST dataset of reconstructed frames. For this purpose you will need to create a txt file from each binary file. Txt file should have image width and height in the first line (separated with space), and then each line should have one event with values separated with space. The data format was discussed during 3rd class. Then, each txt file should be zipped. Then, you can use FireNet repository to generate reconstructions from each zip file. Use 50 ms time windows. Reconstructed images can be used as neural network input as well as other forms of representations. Remember, that folder structure should be preserved.



5 — Classification with DCNN

5.1 Theoretical part

Today we will use the database generated last week to train a neural network for classification. We consider a model that takes a picture as input and a corresponding classifier as output. Instead of directly defining how the model should perform the task, a large input dataset is used along with the correct annotations. Based the examples, the model "learns" how to correctly classify the image.

A breakthrough in the field of object classification was the use of neural networks – a highly simplified model of the brain. They consist of "neurons" – elements that process information. Each has its own inputs, parameters (so-called weights) and one output. The entire model is built from a large number of neurons that are properly connected and arranged in layers (Figure 5.1).

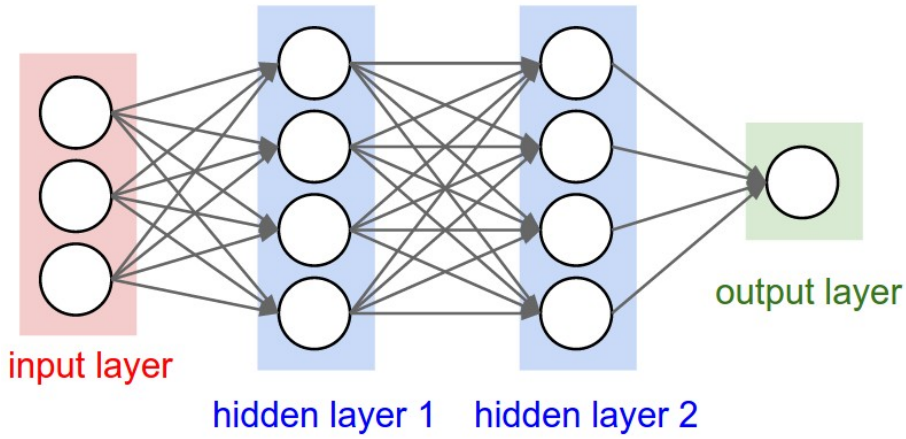


Figure 5.1: Neural network

The way such an architecture works in the context of classification is as follows:

1. The event frame representation, interpreted as a matrix of pixel values is loaded as inputs to the first layer of neurons.
2. Each neuron determines the value of the output based on linear transformations performed on the inputs and weights.

3. The output of the last layer of neurons is a classifier (e.g., a probability matrix of image membership in the correct class). The resulting output is compared with the correct class found in the database. Based on the programmer-defined penalty function, the behavior of the network is evaluated.
4. The goal of network training is to reduce the established penalty function. To do this, the model determines the gradient to calculate the direction of its minimization. The gradient value is then propagated upward in the model (gradient ascent) and the weights in each neuron are updated. It is this mechanism that causes the network to improve its efficiency in each iteration.

Today, the absolute leading tool for classification is convolutional neural networks. Instead of using every pixel value as input to any neuron, convolutional layers consist of matrices that are much smaller in size than the image. These matrices are called filters, and their values are the weights of the model. During image processing, the filter is matched to the first area of the photo, and the values of its weights are multiplied by the pixel values. The same filter is then moved around the entire image generating further results, which, arranged in a matrix, constitute the input to subsequent layers. At the end of the whole model, analogous to traditional neural networks, there is a layer whose output is a classifier (figure 5.2). By using convolutional layers instead of the traditional all-connected layers, the number of parameters in the model has been greatly reduced.

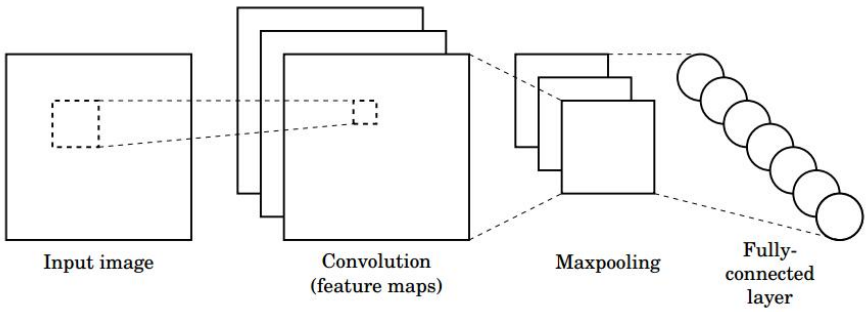


Figure 5.2: Convolutional neural network

During the class, we will use a simple neural network built with several layers. The dataset used is characterized by low complexity, so we will not need a particularly deep neural network. Three convolution layers would be enough. However, combining a large number of linear transformations makes no sense - a linear combination of filter matrices can be replaced by a single matrix. Consequently, the whole network would be just a linear transformation of the input values. Therefore, it is necessary to add some nonlinearity to the model - activation layers that connect successive linear layers. A very popular and simple solution is the use of ReLu activation. They perform the operation f(x) = max(0, x) on the supplied data.

In addition, we will also use MaxPooling layers, which are designed to reduce the dimension of the outputs of a layer, in order to minimize the number of parameters. At the end of our network we need a fully-connected layer. We will use the softmax transformation, which is charateristic for the classification task.

5.2 Practical part

During this class we will use Python and dataset, which last week was generated by reconstructing events from the N-MNIST.

Basic Exercise 5.1 Import datset ■

1. Create Python file and import TensorFlow library as `tf`. You will also need `numpy`, `cv2` and `matplitlib.pyplot` libraries.
2. Use `tf.keras.utils.image_dataset_from_directory` function to import training dataset created last week. You will need to pass a path to `Train` folder. Save the dataset to `train_ds` variable.
3. Use `tf.keras.utils.image_dataset_from_directory` function to import validation dataset created last week. You will need to pass a path to `Test` folder. Save the dataset to `val_ds` variable.

   Protip
   You should also set `image_size` and `batch_size` arguments for this function. The image size is (34,34) and the batch size should be 32. In case of any problems, just google it.

4. Run the code and show the source code to your teacher.

Basic Exercise 5.2 Define and compile the model ■

1. In order to train a DCNN we need to define it first.
2. We will use `tf.keras.Secquential` function to define a model as described in Theoretical part of this class.

   Listing 5.2.1 — CNN model definition:

```
model = tf.keras.Sequential([
  tf.keras.layers.Rescaling(1./255),
  tf.keras.layers.Conv2D(32, 3, activation='relu'),
  tf.keras.layers.MaxPooling2D(),
  tf.keras.layers.Conv2D(32, 3, activation='relu'),
  tf.keras.layers.MaxPooling2D(),
  tf.keras.layers.Conv2D(32, 3, activation='relu'),
  tf.keras.layers.MaxPooling2D(),
  tf.keras.layers.Flatten(),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dense(num_classes,  activation='softmax')
])
```

3. After defining model layers and connections we need to compile it with `model.compile()` function. Google it and study its arguments. We should use `adam` optimizer, `['accuracy']` metric and `tf.keras.losses.SparseCategoricalCrossentropy` loss.
4. Run the code and show the source code to your teacher.

Basic Exercise 5.3 Train and save the network. ■

1. Now its time for CNN training. We do that with `model.fit()` function. Google it and study its arguments. We should use `train_ds` as training dataset, set validation data `validation_data=val_ds`, number of epoch `epochs=5` and batch size `batch_size=32`.

   Protip
   You should save the history of training by using an output of fit function. Just use `history = model.fit(...)`.

2. Save the network with `model.save('path/to/location')`.
3. Use following code to visualise the network training progress.

Listing 5.2.2 — Showing the training progress charts.:

```
# list all data in history
print(history.history.keys())

# summarize history for accuracy
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()

# summarize history for loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```

4. Run the code.
5. It will take some time to train this network. In the meantime find out what is and epoch and what does batch_size mean. Describe it as comment to your code. Then - answer this question: What is the number of iterations per epoch and why?
6. Show the source code to your teacher.

Basic Exercise 5.4 Classify event frame representation. ■

1. Create another Python file, import libraries, define the model and load it with `model.load()`.
2. Load any event frame representation from your `Test` folder. For this purpose use `cv2.imread()` function.
3. Use following code to predict the probability of this image belonging to each of the classes. Print the results.

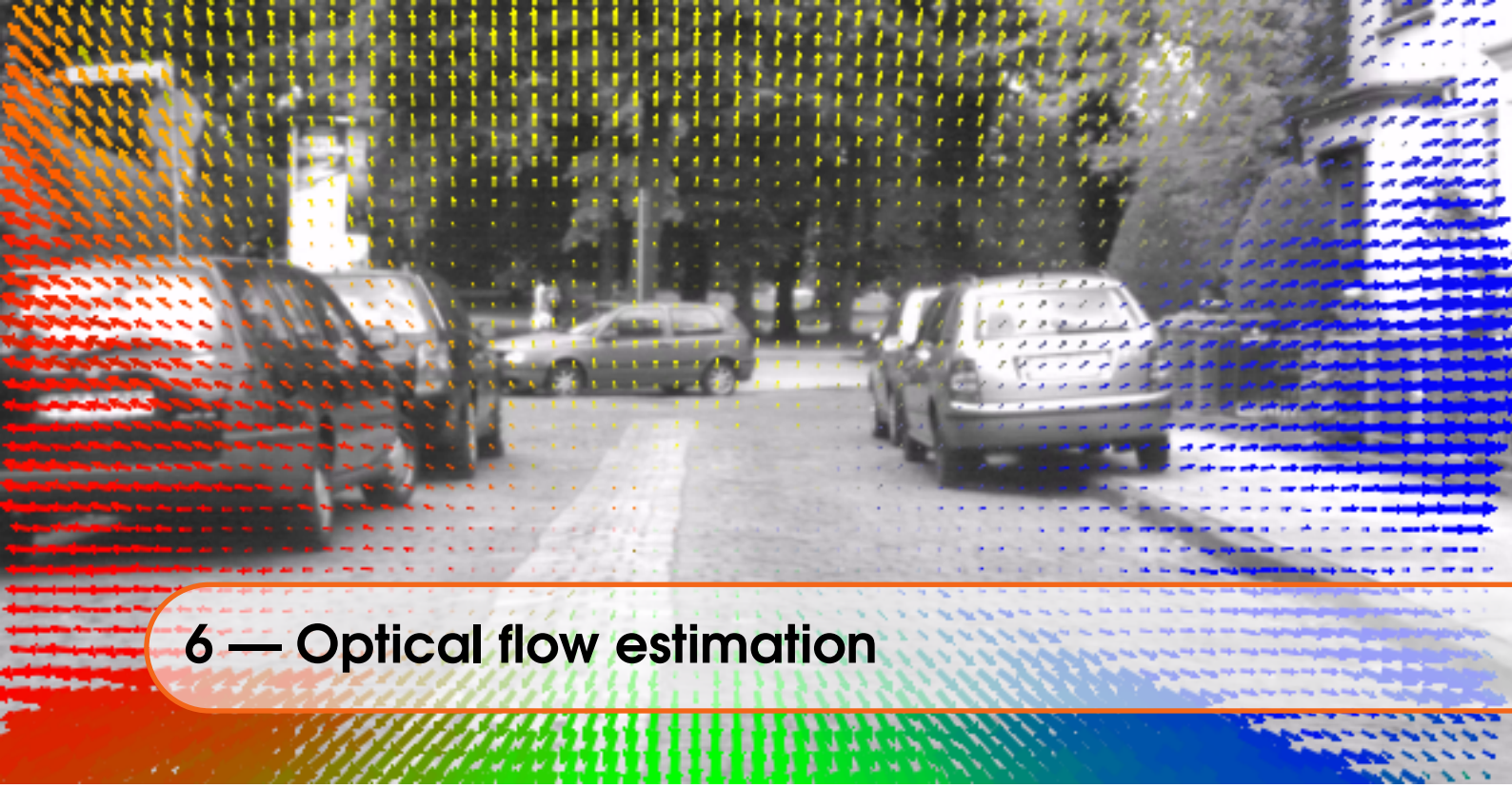
Listing 5.2.3 — Predict the number.:

```
# Predict for random image
image = cv2.imread("./Train/DATASET/8/00032_4.png")
image_to_pred = image.reshape(1,34,34,3)
predict = model.predict(image_to_pred, batch_size=1)
for x in range (10):
        value = float(predict[0][x])
        print("The probaility that the number is " + str(x) + "
    equals " + str(value*100))
```

4. Show image with `cv2.imshow()`.
5. Verify the predictions and compare it to the image.
6. Show the source code to your teacher.

Extension Exercise 5.1 Data augmentation ■

Find out what a data augmentation is and apply it to this data process. Use `RandomFlip` and `RandomRotation` and train the network again (for bigger number of epochs - for example 10). Compare results.



6 — Optical flow estimation

6.1 Obligatory literature

G. Gallego, H. Rebecq and D. Scaramuzza, "A Unifying Contrast Maximization Framework for Event Cameras, with Applications to Motion, Depth, and Optical Flow Estimation," 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 3867-3876, doi: 10.1109/CVPR.2018.00407. Link to paper

6.2 Theoretical part

Optical flow is the pattern of apparent motion of objects, surfaces, and edges in a the scene caused by the relative motion between a camera and a scene. There are many algorithms for determining the optical flow between two frames of traditional video recording. In today's class, we will lean into determining optical flow for event data.

We distinguish between several types of optical flow. Sparse optical flow gives the flow vectors of some "interesting features" (say few pixels depicting the edges or corners of an object) within the frame while Dense optical flow, which gives the flow vectors of the entire frame. Today, however, we will deal with global optical flow, which assumes that all edges are moving with the same velocity on the image plane.

I encourage you to start by reading the paper found in the chapter Obligatory literature. The method introduced by the authors involves estimating the optical flow of a scene based on contrast maximization. The method consists of following steps:

1. First, we aggregate event data during the defined time window.
2. We then take an initial guess of the speed of the scene moving in the x and y planes and write it as θ.
3. Next, we transform each event taking into account its timestamp and velocity θ in such a way as to determine the position of each point at the end of the time window.

$$x_k^{'} = x_k - (t_k - t_m)\theta$$

$$y_k^{'} = y_k - (t_k - t_m)\theta$$

Where $x_k^{'}$ is transformed coordinate, x_k is recorded coordinate, t_k is recorded timestamp and t_m is the last timestamp in sequence.

4. Using the transformed events, we create a frame representation `H` in which the value of a given pixel is the number of transformed events occurring for the given coordinates.
5. Finally, we compute the variance of `H`, which is a function of θ

$$f(\theta) = \sigma^2(H(x;\theta))$$

6. We use the variance of H metric, which is known as contrast in image processing terminology, and we seek to maximize it. The highest contrast of such a generated representation, the better the optical flow velocity estimation. An optimization algorithm, such as gradient ascent or Newton's method, is applied to obtain the best model parameters, i.e., the point trajectories on the image plane that best explain the event data.

Using this simple method, we can estimate the speed of movement of the scene relative to the camera. Moreover, the image of warped events `H` associated to the optimal parameters is a motion-corrected edge-like image, where the "blur" (trace of events) due to the moving edges has been removed.

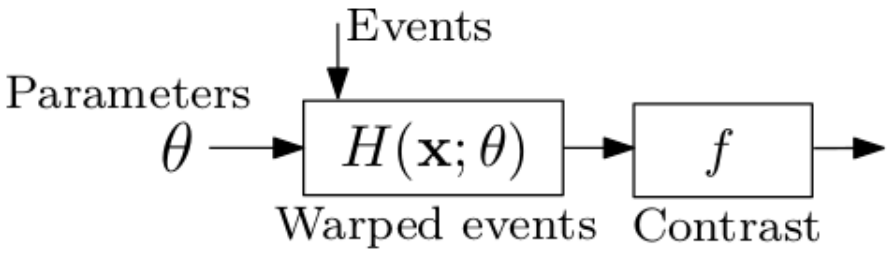


Figure 6.1: Steps for optical flow estimation

6.3 Practical part

During this class we will use Python and DAVIS 240C dataset. Read about it on this website: `https://rpg.ifi.uzh.ch/davis_data.html`. You can download dataset yourself with this link: `https://rpg.ifi.uzh.ch/datasets/davis/shapes_rotation.zip`

Basic Exercise 6.1 Set up dataset, generate event frame representation ■

1. Open (extract) the dataset folder from `/home/Downloads/shapes_rotation`.
2. Create a Python file (use Visual Studio Code). Save it to the same folder, where the `events.txt` is located.
3. Import `cv2` and `numpy` libraries - we are going to need those during this class.
4. Read `events.txt` and save it to a variable.
5. Aggregate events in 100ms time windows and generate event frame representation for each sequence (you can reuse code from Class 2).
6. After each iteration, clear temporary lists and show generated representation with `cv2.imshow()` and `cv2.waitKey()`. (you can reuse code from Class 2).
7. Study movement of the scene in relation to the camera in the 2nd second of the recording (from timestamp 1.0 to timestamp 2.0).
8. Show the source code and generated files to Your teacher.

Basic Exercise 6.2 Create contrast function ■

Listing 6.3.1 — Define following function and fill in the blanks::

```
def contrast(params, xs, ys, ts, ps, image_shape):
    t_max = max(ts)
    h_image = \\TODO Create image of image_shape
    for i in range(len(xs)):
        x_wraped = \\TODO calculate transformed X based on speed params[0]
        y_wraped = \\TODO calculate transformed Y based on speed params[1]
        if \\TODO transformed X and Y are inside the image_shape :
            h_image[y_wraped, x_wraped] += 1
    value = -1 * \\TODO variance of h_image
    return value
```

Protip
Remember to use (-1 x varience) inside the function. In the next exercise we will be looking for the velocity value for which the contrast value is the highest. However, we will use the methods of looking for the minimum of a function - hence -1.

Attention!
You should use $1000000 * params[0]$ instead of $params[0]$ during multiplication of $(t_k - t_m)\theta$. Timestamp is expressed in seconds and important differences are of the microseconds level. In this way, you will simplify the task of minimizing the value of the function depending on the speed.

Basic Exercise 6.3 Estimate optical flow ■

1. Let's go back to the code from exercise 1. After generating event frame, for each aggregated sequence we need to find optimal optical flow speed. For this purpose we'll use `scipy.optimize.fmin` function.

   Listing 6.3.2 — Finding the minimum of contrast function:

   ```
   args = (xs_temp, ys_temp, ts_temp, ps_temp, (180, 240))
   argmax = fmin(contrast, (0,0), args=args, disp=False)
   ```

2. Study the code above. We pass to `scipy.optimize.fmin` the `contrast` function, the fist velocity estimation `(0,0)`, input arguments and `disp=False`. Then, the optimal speed value in x and y coordinates `argmax` is returned.

   Protip
   Find out more about `scipy.optimize.fmin` function here.

3. For each sequence show event frame representation and print the optimal speed value. In addition, also display the image of warped events `H` associated to the optimal parameters (motion-corrected edge-like image, where the "blur" (trace of events) due to the moving edges has been removed).
4. Compare the correctness of estimated optical flow.
5. What is the unit of speed in such a prepared task? Answer as comment to this code.
6. Show the source code and generated files to Your teacher.

Extension Exercise 6.1 Faster function optimization. ■

In the exercise 3 we used very simple optimization technique which uses only function values, not derivatives or second derivatives. Study SciPy documentation here and update exercise 3 with more efficient optimizer. Compare the time of each iteration for both approaches.

7 — Event-by-event processing for tracking

7.1 Obligatory literature

Delbruck, Tobi, and Patrick Lichtsteiner. "Fast sensory motor control based on event-based hybrid neuromorphic-procedural system." 2007 IEEE international symposium on circuits and systems. IEEE, 2007. Link to paper

7.2 Theoretical part

During all the exercises so far, we have used some kind of event data representation to perform vision tasks. Today we will change this assumption and try event-by-event event data processing.

Visual Object Tracking is one of the principal challenges in Computer Vision, where the task is to locate a certain object in all frames of a video, given only its location in the first frame. This is one of the vision tasks for which event cameras are widely used because of their energy efficiency (only changes in the scene are recorded by the camera, which is exactly the data we need for tracking).

In the literature, you will find many implementations of tracking using event data (not infrequently in fusion with traditional camera data). Today we will try to implement our own, very simple solution. We will use a dataset in which objects are stationary relative to each other but move relative to the camera. The size of the objects is also invariant in the set. The method consists of following steps:

1. First, we use traditional video frame to define the objects to be tracked. We define those as state of the scene and each object is described with its centroid coordinates and its size (radius).
2. For each event, we first assign it for one of the objects. We do so by finding the minimal distance between new event coordinates and objects. Events that are not registered near any object are treated as noise.
3. Next, we update selected object state – its position, orientation and size (during this classes we will simplify this problem and only update object position).
4. We repeat the process described in this way (steps 2-3) for each new registered event.

Using this simple method, we can track multiple object at once. We track only their position with no regard to changes in size or rotations.

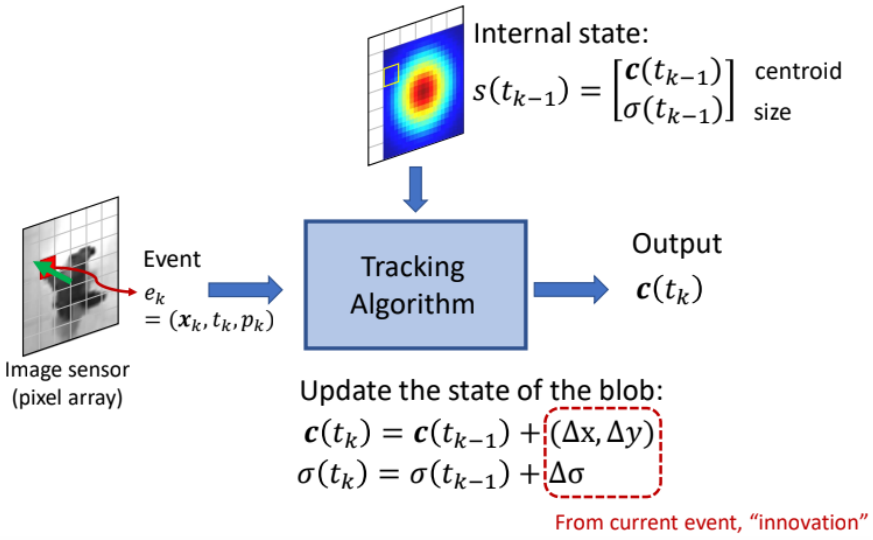


Figure 7.1: Event-by-event object tracking

7.3 Practical part

During this class we will use Python and DAVIS 240C dataset. Read about it on this website: `https://rpg.ifi.uzh.ch/davis_data.html`. You can download dataset yourself with this link: `https://rpg.ifi.uzh.ch/datasets/davis/shapes_rotation.zip`

Basic Exercise 7.1 Initialize objects based on video frame ■

1. Open (extract) the dataset folder from `/home/Downloads/shapes_rotation`.
2. Create a Python file (use Visual Studio Code). Save it to the same folder, where the `events.txt` is located.
3. Import `cv2` and `numpy` libraries - we are going to need those during this class.
4. Read `images/frame_00000022.png` image with `cv2.imread()`. The image is grayscale so you should use `cv2.IMREAD_GRAYSCALE` parameter. We use this image, because we will start tracking at timestamp equal to 1 second and this image corresponds to that time.
5. Use `cv2.SimpleBlobDetector` to detect objects in the scene. First, you are going to need to change parameters.

Listing 7.3.1 — SimpleBlobDetector:

```
# Setup SimpleBlobDetector parameters.
params = cv2.SimpleBlobDetector_Params()

# Change thresholds
params.minThreshold = 10;
params.maxThreshold = 200;

# Filter
params.filterByArea = True
params.minArea = 100
params.filterByCircularity = False
params.filterByConvexity = False
```

```
ver = (cv2.__version__).split('.')
if int(ver[0]) < 3 :
    detector = cv2.SimpleBlobDetector(params)
else :
    detector = cv2.SimpleBlobDetector_create(params)
```

6. Use `detector.detect()` function to detect objects on the image. Then, use the `cv2.drawKeypoints` to draw detected blobs as red circles.
7. Display image with `cv2.imshow()` and `cv2.waitKey()`
8. Each object is defined with its centroid coordinates and diameter of the circle. Print the coordinates and radius of each detected object and save each object as element in the list.

```
objects = []
for keypoint in keypoints:
        objects.append([int(keypoint.pt[0]), int(keypoint.pt[1]), int(
    keypoint.size)])
```

9. Show the source code and generated images/prints to Your teacher.

Basic Exercise 7.2 Track all object with simple tracker ■

1. Read `events.txt` and save it to a variable (from timestamp 1.0 to timestamp 5.0).
2. Loop through all saved events and for each of them:
   - Calculate Euclidean distance between the new event and each object centroid. We assume that the event belongs to the object from which it has the smallest distance

     Attention!
     You can do so with `math.dist()` function.

   - If calculated smallest distance is smaller then the radius of the object - update object centroid coordinates - move it in x and y planes in the direction of new event by half the difference of centroid and event coordinates.

     ```
     if (distance < object_diameter/2 and distance > 0):
             new_center_x = old_center_x + (event_x - old_center_x)/2
             new_center_y = old_center_y + (event_y - old_center_y)/2
     ```

   - If calculated smallest distance is bigger then the radius of the object - ignore it. We will treat those events as noise.
3. In order to visualize object - aggregate events in 10ms time windows and generate event frame representation for each sequence (you can reuse code from Class 2).
4. After each iteration (every 10ms), use `cv2.circle()` function to draw a circles with updated object centroids and radius.
5. Clear temporary lists and show generated representation with `cv2.imshow()` and `cv2.waitKey()`.
6. Show the source code and generated files to Your teacher.

Basic Exercise 7.3 Track single object with another tracker ■

1. Update code from exercise 1 to use only one object. You can choose any one you want. In this task we will track only single object.
2. In this task, we will determine the centroid of an object as the average value of the last 10 values of the coordinates of events located in the neighborhood of the event. Define `X_list` and `Y_list` lists and append object centroid coordinates.
3. Loop through all saved events and for each of them:

- Calculate Euclidean distance between the new event and selected object centroid.
- If calculated smallest distance is smaller then the radius of the object - update object centroid coordinates. Add new x and y values to the lists, and update new centroid coordinates as an average of values stored in `X_list` and `Y_list`.

```
if (distance < object_diameter/2 and distance > 0):
        new_center_x = int(sum(X_list)/len(X_list))
        new_center_y = int(sum(Y_list)/len(Y_list))
```

- If calculated smallest distance is bigger then the radius of the object - ignore it. We will treat those events as noise.
- If `X_list` and `X_list` length is bigger than 10, delete first (oldest) value.

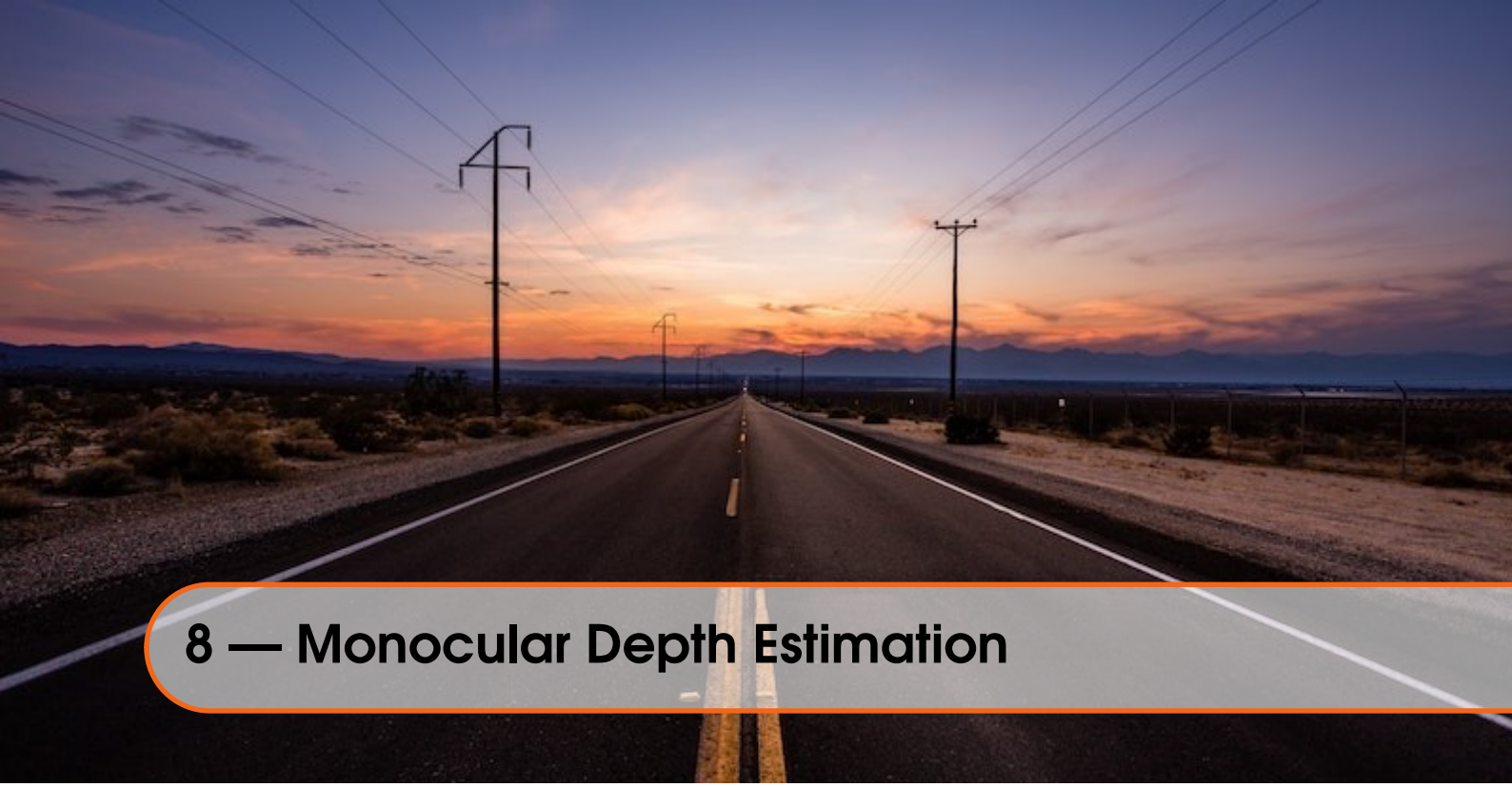
> **Attention!**
> You can do so with `del X_list[0]` command.

- Save each updated object centroid coodrinates to separate list (`X_centers` and `Y_centers`)

4. In order to visualize object path - aggregate events in 10ms time windows and generate event frame representation for each sequence (you can reuse code from Class 2).
5. For each coodrinate in `X_centers` and `Y_centers` list use `cv2.circle()` function with `radius=0` and `thickness=1` to draw a point for each updated object centroid coordinates. In this way, you will obtain a drawing of the path traveled by the object.
6. Clear temporary lists and show generated representation with `cv2.imshow()` and `cv2.waitKey()`.
7. Show the source code and generated files to Your teacher.

Extension Exercise 7.1 Try different approach ■

The approach we choose is very simple and limited. Try to implement one of the approaches introduced in literature. This may be one of the following: Asynchronous Event-Based Multikernel Algorithm for High-Speed Visual Features Tracking, Feature Detection and Tracking with the Dynamic and Active-pixel Vision Sensor (DAVIS), Visual Tracking Using Neuromorphic Asynchronous Event-Based Cameras or Human vs. computer slot car racing using an event and frame-based DAVIS vision sensor



8 — Monocular Depth Estimation

8.1 Obligatory literature

Hidalgo-Carrió, Javier, Daniel Gehrig, and Davide Scaramuzza. "Learning monocular dense depth from events." 2020 International Conference on 3D Vision (3DV). IEEE, 2020. Link to paper

8.2 Theoretical part

Monocular depth estimation is the task of estimating the depth of objects in an image using only information from a single camera. The goal is to infer the relative distance of different objects in the scene based on their appearance in the image. This is generally a more challenging problem than stereo depth estimation, which uses information from two cameras to triangulate the depth of objects in the scene. Monocular depth estimation algorithms typically rely on machine learning techniques, such as deep neural networks, to infer depth from image data.

Dense depth estimation and sparse depth estimation are two different approaches to estimating the depth of objects in an image or a video. Dense depth estimation aims to estimate the depth of all pixels or voxels in an image or a video. This approach is typically used for tasks such as 3D reconstruction, where a detailed 3D model of the scene is desired. Sparse depth estimation, on the other hand, aims to estimate the depth of only a subset of pixels or points in an image or a video. This approach is typically used for tasks such as object detection or tracking, where only the depth of a specific object or feature is needed.

One approach to dense monocular depth estimation with event cameras is to use a deep neural network to learn the relationship between the event data and the depth of the objects in the scene. The network can be trained on a dataset of event data and corresponding depth maps, and then applied to new event data to estimate depth.

Another approach is to use the event data to estimate the optical flow of the scene, which encodes information about the motion of objects in the scene. The relative motion of objects can be used to infer their relative depth.

Event cameras are still relatively new technology and monocular depth estimation with it is still an active area of research. There are many other methods and techniques being developed to use event cameras for depth estimation.

8.3 Practical part

During this class we will use Python and DENSE dataset. Read about it on this website: `https://rpg.ifi.uzh.ch/E2DEPTH.html`.

Basic Exercise 8.1 Monocular Depth Estimation with event camera ■

1. Open `https://github.com/uzh-rpg/rpg_e2depth` repository, read `README`.
2. Clone repository on Your hard drive (use `git clone` function).
3. Download pretrained neural network model and testing dataset with functions introduced in `README` file.
4. Run the monocular depth estimation with following code. Analyze the meaning of all used input arguments.

```
python3 run_depth.py -c pretrained/E2DEPTH_si_grad_loss_mixed.pth.tar \
  -i data/test/events/voxels \
  --output_folder /tmp \
  --save_numpy \
  --show_event \
  --display \
  --save_inv_log \
  --save_color_map
```

5. While running the example, use sliders to change parameters. Explain the meaning of each parameter that can be changed with sliders.
6. Show the source code, answers and the generated sequences to your teacher.

Basic Exercise 8.2 Research different depth estimation approaches ■

1. Event cameras are still relatively new technology and an active area of research. Our last task is to research some event cameras applications. Let's focus on the topic of today's classes - depth estimation. Use the following links:
   - `https://github.com/uzh-rpg/event-based_vision_resources` - the list of the event-based vision resources (papers, links to datasets, tools, repositories)
   - `https://rpg.ifi.uzh.ch/research_dvs.html` - website related to event camera research conducted by the University of Zurich
   - `https://rpg.ifi.uzh.ch/docs/EventVisionSurvey.pdf` - survey paper of event cameras applications and algorithms.
2. Research different approaches fordepth estimation with event cameras. Write a short documents (1-2 A4 pages) compering at least 2 different algorithms.
3. Upload document to UPEL and show it to your teacher. Explain researched approaches.

9 — From video frames to DVS events

9.1 Obligatory literature

Hu, Yuhuang, Shih-Chii Liu, and Tobi Delbruck. "v2e: From video frames to realistic DVS events." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2021. Link to paper

9.2 Theoretical part

Work on the use of Dynamic Vision Sensors for perception tasks that are applicable to autonomous vehicles such as cars and drones is made difficult by the small number of available datasets containing the labled event data sequences. However, this problem can be solved by using tools to generate realistic event data from a classic video sequence. In today's class, we will use v2e to generate event data from a video.

The tool used implements event generation in the following steps:

1. RGB frames are converted to luma frames.
2. Luma frames are resized to output size of selected DVS that is being simulated.
3. The luma frames are then interpolated using the `Super-SloMo` video interpolation network (Based on the determined optical flow (estimation of the direction of movement of objects), the number of frames per second is increased with interpolation). You can find more information about this network in this paper - *H. Jiang, D. Sun, V. Jampani, M. Yang, E. Learned-Miller, and J. Kautz. Super SloMo: High quality estimation of multiple intermediate frames for video interpolation. In 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 9000–9008, 2018 Link to paper*
4. The linear to logarithmic mapping is applied because standard digital video usually represents intensity linearly, but DVS pixels detect changes in log intensity. Than, a finite intensity-dependent photoreceptor bandwidth filter is applied (read more in paper).
5. Based on upsampled video the event data is generated.
6. Some temporal noise characteristic for DVS is added.
7. The achieved event data is saved along with input video resized and converted to luma, Slow Motion video and event-frame representation video.

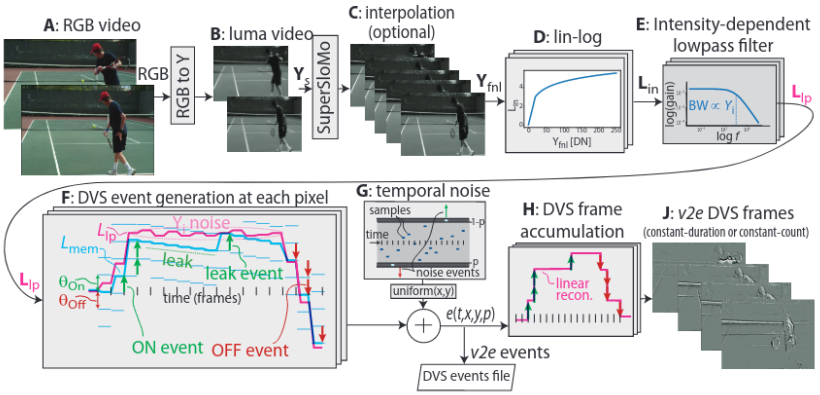


Figure 9.1: Steps for v2e event generation

9.3 Practical part

During this class we will use Python, videos supplied by v2e repository and custom video created during classes.

Basic Exercise 9.1 Set up v2e, run example ■

1. Study the v2e tools - You'll find information on v2e webside, repository and paper.
2. Clone `https://github.com/SensorsINI/v2e` repository. Read `README` file.
3. Install necessary libraries and packages. You'll find information on how to do so in `README` file.

> **Protip**
> You can use `pip3 install -e .` to install all necessary packages.

4. Download pretrained `Super-SloMo` network and place it in `input` directory (You'll find it in `README` file).
5. Download sample `tennis` video and place it in `input` directory (You'll find it in `README` file).
6. Study v2e python script arguments.
7. Run example with following code:

Listing 9.3.1 — v2e example:

```
python v2e.py -i input/tennis.mov --overwrite --timestamp_resolution
    =.003 --auto_timestamp_resolution=False --dvs_exposure duration
    0.005 --output_folder=output/tennis --overwrite --pos_thres=.15
    --neg_thres=.15 --sigma_thres=0.03 --dvs_aedat2 tennis.aedat --
    output_width=346 --output_height=260 --stop_time=3 --cutoff_hz=15
```

8. Describe all of generated files in comments for Your code.
9. Show the source code and generated files to Your teacher.

Basic Exercise 9.2 Use v2e to generate events for custom video ■

1. Record Your own short video. You can record whatever You want. The only requirement is the dynamics of the scene - make sure that Your video contain movement (it can be fast).
2. Save the recorded video in `input` directory.

3. Run v2e for Your custom video with following parameters:
    - In order to make the process of generating output files shorter - make the output of the video smaller then the input (with the proportions of the width and height of the input video). You can do it with `output_height` and `output_width` parameters.
    - During our classes we use events saved in `txt` format, not `aedat2`. Disable `aedat2` output (with `dvs_aedat2` parameter) and enable `txt` output (with `dvs_text` parameter.
    - Disable preview with `no_preview` parameter.
    - You can leave other parameters as default. If you choose to use some other parameters (in order to shorten the process of generation, set start and stop time for video or any other reason) - describe them as comment.
4. Describe all of generated files as comments for Your code.
5. Open generated `txt` file with event data and study its content.
6. Reuse code from first class to represent part of the generated sequence as 3D chart (10ms will be enough). Explain why the generated events look this way (as comment).
7. Show the source code and generated files to Your teacher.

Extension Exercise 9.1 Event-frame representation for generated event data ■

Open generated `txt` file with event data and study its content. Then, use this file to create event representation of generated data. You can reuse the code from previous classes. Set accumulation time window according to Your preference (consider how the timestamp is represented in the generated data). You can use any representation you want.

10 — Hybrid event cameras, data fusion

10.1 Obligatory literature

Wang, Ziwei, et al. "An asynchronous kalman filter for hybrid event cameras." Proceedings of the IEEE/CVF International Conference on Computer Vision. 2021. Link to paper

10.2 Theoretical part

In our classes so far, we have used event cameras for several applications, taking advantage of its strengths over traditional cameras (high dynamic range, low latency, elimination of redundant information, etc.). It should be remembered, however, that traditional cameras also have advantages that are crucial for some applications (precise information on pixel brightness values). For many computer vision tasks, the best results in terms of accuracy can be obtained through the fusion of event data and classic video frames. Some event cameras are hybrid - they record both types of data in parallel.

In today's class, we will use a dataset containing both video frames and event data, and our goal will be to generate a video sequence characterized by High Dynamic Range and low motion blur. Let's start our work by reading the obligatory literature attached above. We will use described algorithm with an asynchronous Kalman filter during today's classes.

The proposed image processing architecture is shown in Fig. 10.1 There are three modules in the proposed algorithm; a frame augmentation module that uses events to augment the raw frame data to remove blur and increase temporal resolution, the Asynchronous Kalman Filter (AKF) that fuses the augmented frame data with the event stream to generate HDR video, and the Kalman gain module that integrates the uncertainty models to compute the filter gain. The potential of such algorithms to enhance conventional video to overcome motion blur and increase dynamic range has applications from robotic vision systems (e.g., autonomous driving), through film-making to smartphone applications for everyday use.

10.3 Practical part

During this class we will use Matlab, Python and *city_09d* dataset. Read about it on this website: `https://github.com/ziweiWWANG/AKF`. You can download dataset yourself with this link. We are going to need Matlab online and Matlab drive for this class.

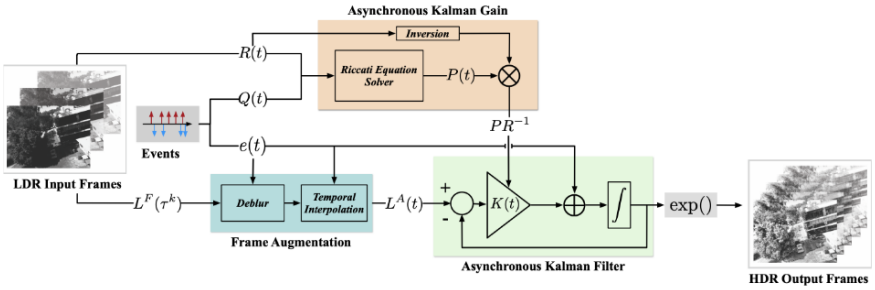


Figure 10.1: Asynchronous Kalman Filter pipeline

Basic Exercise 10.1 Apply AKF algorithm with Matlab ■

1. Clone `https://github.com/ac-freeman/AKF` repository. Read through README file.
2. Inside the repository, you will find multiple *.m* files. In our classes so far, we have used Python. However, many researchers use the C/C++ and Matlab languages to work with event data. Today we will try to use the code provided by the authors of the publication in Matlab.
3. Open this link. Sign up with your university email and open Matlab online.
4. Open this link. Sign up with your university email and open Matlab drive.
5. Download *city_09d_150_200.mat* dataset from this link
6. Upload all *.m* files to Matlab drive. Create directory *data* and upload dataset to this directory.
7. Open *run_akf* script in Matlab online. Study selected parameters in README file. Leave all parameters as default.
8. Run *run_akf* script in Matlab online (just type script name in Command Window (at the bottom of the page)). It will take some time.
9. Study generated files in *reconstructed* directory. Download it with Matlab Drive and save on your PC.
10. Show the source code and generated files to Your teacher. Describe it.

Basic Exercise 10.2 Visualize High Dynamic Range video ■

1. Create a Python file (use Visual Studio Code). Save it to the same folder, where the *city_09d_150_200.mat* is located.
2. Copy *reconstructed* directory to the same folder, where the Python file is located.
3. Our task is to visualize original video frames (stored in *.mat* dataset), event-frame representation of original event data (stored in *.mat* dataset) and generated High Dynamic Range output frames (stored in *reconstructed*) at once. We need to synchronize all types of frames to visualize images with the same timestamps.
4. First, read dataset file with *h5py* library. We are going to need events, images and images timestamps.

```
import numpy as np
import h5py

f = h5py.File('city_09d_150_200.mat','r')

images_orginal = f.get('image')
images_orginal = np.array(images_orginal)
```

```
timestamps_images_orginal = f.get('time_image')
timestamps_images_orginal = np.array(timestamps_images_orginal)/1000000

events = f.get('events')
events = np.array(events)
timestamps = events[0]/1000000
x_values = events[1]
y_values = events[2]
pol = events[3]
```

5. Study all created arrays. *images_orginal* should contain original images sequence (in grayscale with the size of 640x480). *timestamps_images_orginal* should contain precise timestamps for each corresponding image. *events* should contain event data. The values of timestamps is expressed in microseconds (we divided it by 1000000 to express it in seconds).
6. Study *reconstructed* directory and *timestamps.txt* file. This file contain the name of each generated HDR image and its precise timestamp.
7. Read *timestamps.txt* file with Python and store HDR images timestamps. There should be a bit less HDR frames than original frames.
8. Read all images in *reconstructed* directory with Python and store them along with their timestamps.
9. Visualize all corresponding original images, HDR images and event - frame representations of event data sequences. You should use timestamps of images to synchronize all inputs (accumulate correct parts of event data, display correct original and HDR images).

   Protip
   Loop through all HDR images timestamps, find corresponding HDR image and original image. You could use Python dictionaries or lists with corresponding indexes for images and timestamps. Use image timestamp to accumulate events (Use events with timestamps from 0 to *HDR_image_timestamp* in the first iteration and events with timestamps from previous *HDR_image_timestamp* to the current *HDR_image_timestamp* for all other iterations).

10. Show the source code and generated files to Your teacher. Describe the HDR images and compare them to original images.

Extension Exercise 10.1 Research event data - image frame fusion applications. ■

Find event data - image frame fusion applications. You can use event vision resources (`https://github.com/uzh-rpg/event-based_vision_resources`) and Google Scholar webside. Write short document (1-2 A4 pages) describing some of the applications found in literature.